

Got it — here is a **complete, clean JPQL-only program** (minimum required files). No native query included. This will run properly.

Uses Spring Boot + JPQL + MySQL.

✓ 1. Main Class

```

</> Java
package com.example.jpqldemo;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class JpqlDemoApplication {
    public static void main(String[] args) {
        SpringApplication.run(JpqlDemoApplication.class, args);
    }
}

```

✓ 2. Entity Class

```

</> Java
package com.example.jpqldemo.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "patient")
public class Patient {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;
    private int age;

    public Patient() {}

    public Patient(String name, int age) {
        this.name = name;
        this.age = age;
    }

    public Long getId() { return id; }
    public String getName() { return name; }
    public int getAge() { return age; }

    public void setName(String name) { this.name = name; }
    public void setAge(int age) { this.age = age; }
}

```

✓ ◆ 3. Repository (JPQL)

```
</> Java   
  
package com.example.jpqldemo.repository;  
  
import com.example.jpqldemo.entity.Patient;  
import org.springframework.data.jpa.repository.*;  
import org.springframework.data.repository.query.Param;  
  
import java.util.List;  
  
public interface PatientRepository extends JpaRepository<Patient, Long> {  
  
    // JPQL Query  
    @Query("SELECT p FROM Patient p WHERE p.age > :age")  
    List<Patient> findPatientsAboveAge(@Param("age") int age);  
}
```

✓ ◆ 4. Controller

◆ Get Plus ×

```
</> Java   
  
package com.example.jpqldemo.controller;  
  
import com.example.jpqldemo.entity.Patient;  
import com.example.jpqldemo.repository.PatientRepository;  
  
import org.springframework.web.bind.annotation.*;  
  
import java.util.List;  
  
@RestController  
@RequestMapping("/api/patient")  
public class PatientController {  
  
    private final PatientRepository repository;  
  
    public PatientController(PatientRepository repository) {  
        this.repository = repository;  
    }  
  
    // Test API  
    @GetMapping("/check")  
    public String check() {  
        return "JPQL API Working";  
    }  
  
    // Insert Data  
    @PostMapping("/add")  
    public String addPatient(@RequestBody Patient patient) {  
        repository.save(patient);  
        return "Patient added!";  
    }  
  
    // JPQL Query API  
    @GetMapping("/above/{age}")  
    public List<Patient> getPatientsAboveAge(@PathVariable int age) {  
        return repository.findPatientsAboveAge(age);  
    }  
}
```



✓ 5. application.properties

```
</> properties

server.port=8082

spring.datasource.url=jdbc:mysql://localhost:3306/testdb
spring.datasource.username=root
spring.datasource.password=1234

spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

✓ 6. pom.xml

```
</> XML

<project xmlns="http://maven.apache.org/POM/4.0.0">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.example</groupId>
  <artifactId>jpqldemo</artifactId>
  <version>0.0.1-SNAPSHOT</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.2.0</version>
  </parent>

  <dependencies>

    <!-- REST -->
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
```

```
</> XML  Get Plus x

  </dependency>

  <!-- JPA -->
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>

  <!-- MySQL -->
  <dependency>
    <groupId>com.mysql</groupId>
    <artifactId>mysql-connector-j</artifactId>
  </dependency>

</dependencies>
</project>
```

✓ Project Structure

```
com.example.jpqldemo
├── JpqlDemoApplication.java
├── controller
│   └── PatientController.java
├── entity
│   └── Patient.java
├── repository
│   └── PatientRepository.java
```

🚀 Test in Postman

✓ Check API

```
GET http://localhost:8082/api/patient/check
```